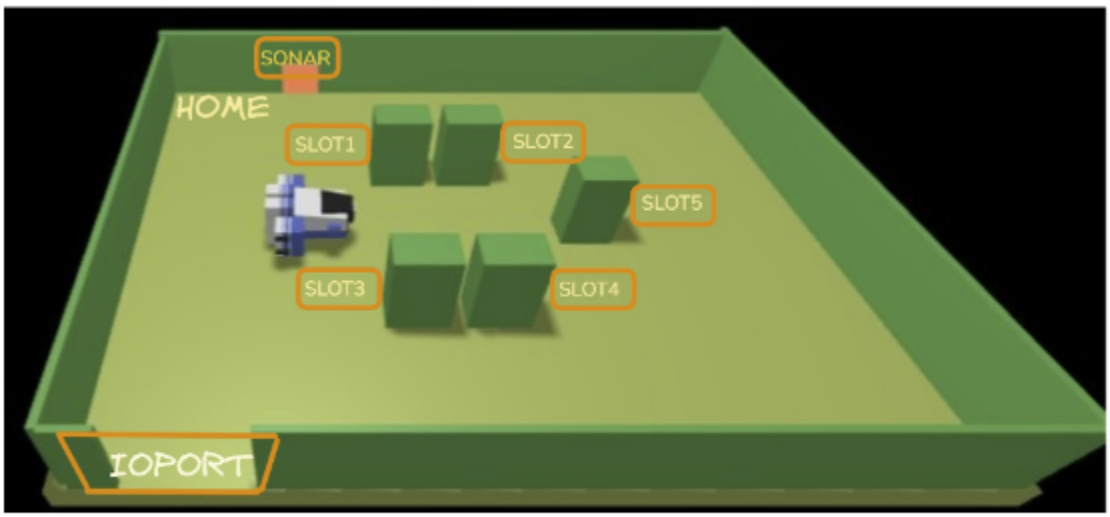
Sprint 0

Marchini Claudio, Tomasi Cesare

Introduction

The original specification for the project is found here, for comodity a copy is reported below.

A *Maritime Cargo shipping company* (from now on, simply **company**) intends to automate the operations of load of **containers** in the ship's cargo hold (or simply hold). To this end, the company plans to employ a *Differential Drive Robot* (from now, called **cargorobot**). The *hold* is a rectangular, flat are a with an Input/Output port (**IOPort**). The area provides `4 slots` to store the containers and a slot named **slot5**.



In the picture above:

- The **slots1-4** depict the hold areas reserved to store one *container* each,
- The **slots5** depicts an area, where the *cargorobot* must temporarily store a container, before to place it in one of the *slots1-4*. During temporary storage, a 'marker' device labels the container with an identification barcode and signals when this marking activity is completed.
- The **IOPort** is a device with a **pushbutton** and a **display**. The *pushbutton* is pressed by the customer in order to to send a request to load a container on the cargo. The *display* is used to show the answer to the request and to show the current state of the *hold*.
- The **sensor** associated to the *IOPort* is a device (a *sonar*) used to detect the presence of a container, when it measures a distance `D`, such that `D < DFREE/2`, during a reasonable time (e.g. `3` secs).

Requirements

The company asks us to build service named **cargoservice** that should work as follows. The *cargoservice* is able to receive a **request to load** a container sent by some customer by using the *pushbutton* of the *IOPort*.

- It sends the answer `retrylater`, if the **IOPort** is currently occupied by a container or if the system is *Out of service*
- It rejects the request when the hold is already full, i.e. the `slots1-4` are already occupied.
- Otherwise, it considers the system as *engaged*, detects a free slot and returns as answer the name of such a reserved slot. While engaged, the system must blink a Led.

When the *request to load* is accepted, the customer must move the container in the *sensor* area within prefixed amount of time (e.g. 30 secs), otherwise the systems becomes **disengaged**. Then, the *cargoservice* uses the *cargorobot* to move the container from the *IOPort* to the *slot5* (for marking the container) and then to the reserved slot. The service must also show on the *display* on the *IOPort*:

- the current state of the *hold*
- the message **'Service working'**, when it is all is going well
- the message **'Out of service'** if the *sonar sensor* measures a distance `D > DFREE` for at least `3` secs (perhaps a failure of the *sonar*).

Requirement Analysis

The system states the presence of a *hold* which contains *slots* and an *IOPort*. Theese objects must me modeled inside the system and will likely be used by the robot to plan its movement across the hold. We begin by defining a *slot*, it must contain:

- An **id** to distinguish it (1-5)
- A **flag** to specify it is currently occupied

```
public interface ISlot {
    public int getID();
    public boolean isOccupied();

    public void setOccupied(boolean value);
}
```

The slots are contained in the hold, which also has the *IOPort* location

```
public interface IPosition {
    public int getX();
    public int getY();
}

public interface IHold {
    public IPosition getIOPortPosition();
    public IPosition getHomePosition();
    public List<Pair<IPosition, ISlot>> getSlots();
}
```

Since the *hold* is rectangular in shape, the reprentation of the map can be a mat...
We choose to make each cell the size of the robot, the position reprents the

coordinates of the cell in this matrix. This system can take advantage of the robot's movement system.

The special *slot5* is distinguished by its ID which is always `5`.

The *cargorobot*'s initial position is defined by the **HOME**, it is formalized as a special position in the hold obtained by `getHomePosition()`

The current requirements do not specify any data about the *container*, the only information needed is to know if it currently occupied a slot, which can be obtained by the method `isOccupied()`, so the current system avoids modeling it.

Since the system will be deployed on multiple nodes that are eterogeneous (Raspberry Pico W, a physical robot and a master node) a distributed model based on services is required. Using a general purpose progamming language (such as Java or C#) for this specific use case would force us to writes lots of lines of code and also would be very technology dependent on the chosen communication protocol. For this reasons, our software house has already developed a DSL for this specific purpose: **qak**. The language is very concise and also supports multiple communication protocols (that will likely be used by the system, like TCP, WebSocket, MQTT) all in a transparent way for the programmer. This makes *qak* the best choice for this project wherever a service is needed.

The requirements present a message called **request to load** from the *pushbutton* to the *cargoservice*, which is used to start the load process. Since we want to model the service as a collection of microservices we will the `qak` language. The message expects an aswer so it will be modeled as a *request*

```
Request loadRequest : loadRequest(X)

Reply retryLater : retryLater(RetryMessage) for loadRequest
Reply rejected : rejected(RejectedMessage) for loadRequest
Reply accepted : accepted(SlotID) for loadRequest
```

The requirements specify that an LED (hardware component which is capable of emitting light) must blink while the system is *engaged*. The company told us that the chosen LED is going to be the one attached to the Raspberry Pico board.

The system requires us to model the **cargorobot**, fortunately the software house has previously built a series of components that are used to command a DDR robot, namely:

- VirtualRobot26
- RobotObj26
- RobotService26
- RobotSmart26

From the requisites whe can deduce that the robot will:

1. Go from *home* to the *IOPort*
2. Go from *IOPort* to *slot5*
3. Wait *3 seconds* for the *marker* to finish (the client specified the wait time is fixed)
4. Go from *slot5* to the reserved slot
5. Go back *home*

Fortunately the implementation **RobotSmart26** has a lot of features that the system need already built in, such as:

- A pathfinding algorithm
- A map based o an grid
- A movement system based on *steps* of fixed sizes (the cells of the grid)

- It is interactable via *qak* messages

The **IOPort** is the component that is used by the client to interact with the system. The company told us that the **pushbutton** for the *IOPort* and the **display** must be a web page that will be used by various users to control the system and watch its status. The page will then need:

- A button that will act as the *pushbutton* to send the *request to load*
- At least some string that show the response and the state of the hold

```
<!DOCTYPE html>
<head>
    <title>IOPort Display</title>
</head>
<body>
    <button>Request to load</button><br>
    Response: <span id="request-result">accepted at hold 1</span><br>
    Current status: <span id="current-status">service working</span>
</body>
</html>
```

Problem Analysis

Test Plans

Since at the current status the system doesn't have much code, the only component that can be tested is the *hold*, in particular:

1. If the hold is empty a *request to load* should give an *accepted* response.
2. If the hold is full a *request to load* should give a *rejected* response.

```
public class HoldTest {
    public void TestEmptyHold() {
        var hold = new Hold();

        assertFalse(hold.getSlots()[0].getValue().isOccupied());
        assertFalse(hold.getSlots()[1].getValue().isOccupied());
        assertFalse(hold.getSlots()[2].getValue().isOccupied());
        assertFalse(hold.getSlots()[3].getValue().isOccupied());
    }

    public void TestFullHold() {
        var hold = new Hold();
        for (int i = 0; i < 4; i++) {
            hold.getSlots()[i].getValue().setOccupied(true);
        }

        assertTrue(hold.getSlots()[0].getValue().isOccupied());
        assertTrue(hold.getSlots()[1].getValue().isOccupied());
        assertTrue(hold.getSlots()[2].getValue().isOccupied());
        assertTrue(hold.getSlots()[3].getValue().isOccupied());
    }
}
```

Project

Testing

Deployment

Maintenance

Marchini Claudio
claudio.marchini@studio.unibo.it

Tomasi Cesare
cesare.tomasi@studio.unibo.it

SignedSnow0/CargoService